Your Institution Name

MINI PROJECT · DELIVERABLE 3

Output Screenshots

Extractive Text Summarisation with TextRank
TF-IDF sentence graph · PageRank · ROUGE-1/2/L evaluation

Student	**Your Full Name**
Roll number	Your Roll Number
Course	Natural Language Processing · Semester —
Guide	Guide Name

Contents

1. Streamlit application — summary, ROUGE metrics and explainability views
2. Jupyter Notebook — preprocessing, outputs and result tables
3. Evaluation charts and ROUGE results table
4. Terminal output of the full experiment

October 2026

1. INTERACTIVE DEMO (STREAMLIT)

Generated summary and live ROUGE

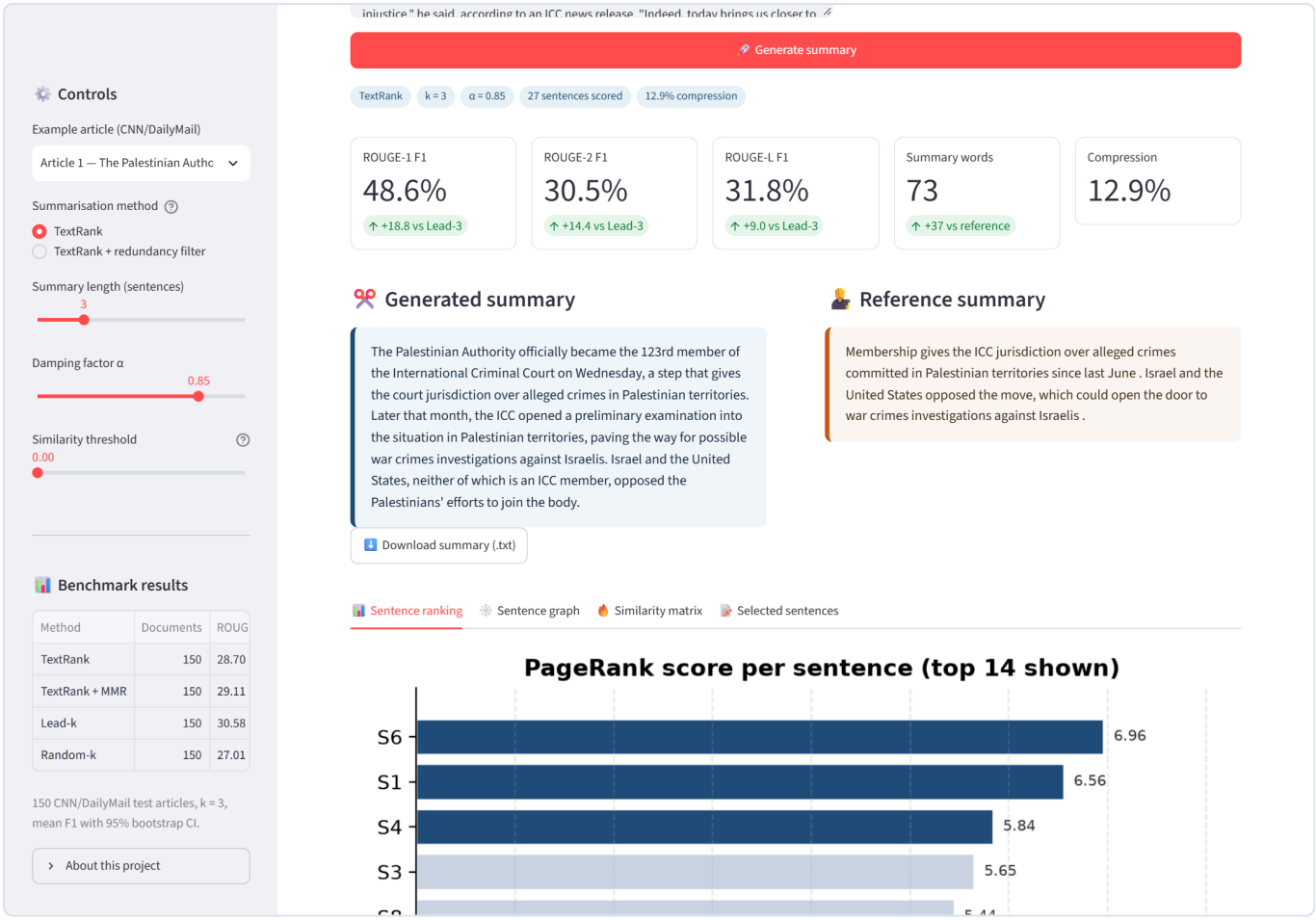


Figure 1. The Streamlit application after summarising a CNN/DailyMail article: generated summary on the left, human reference on the right, and ROUGE-1, ROUGE-2 and ROUGE-L F1 cards with the delta against the Lead-3 baseline.

1. INTERACTIVE DEMO (STREAMLIT)

Full application page (part 1 of 2)

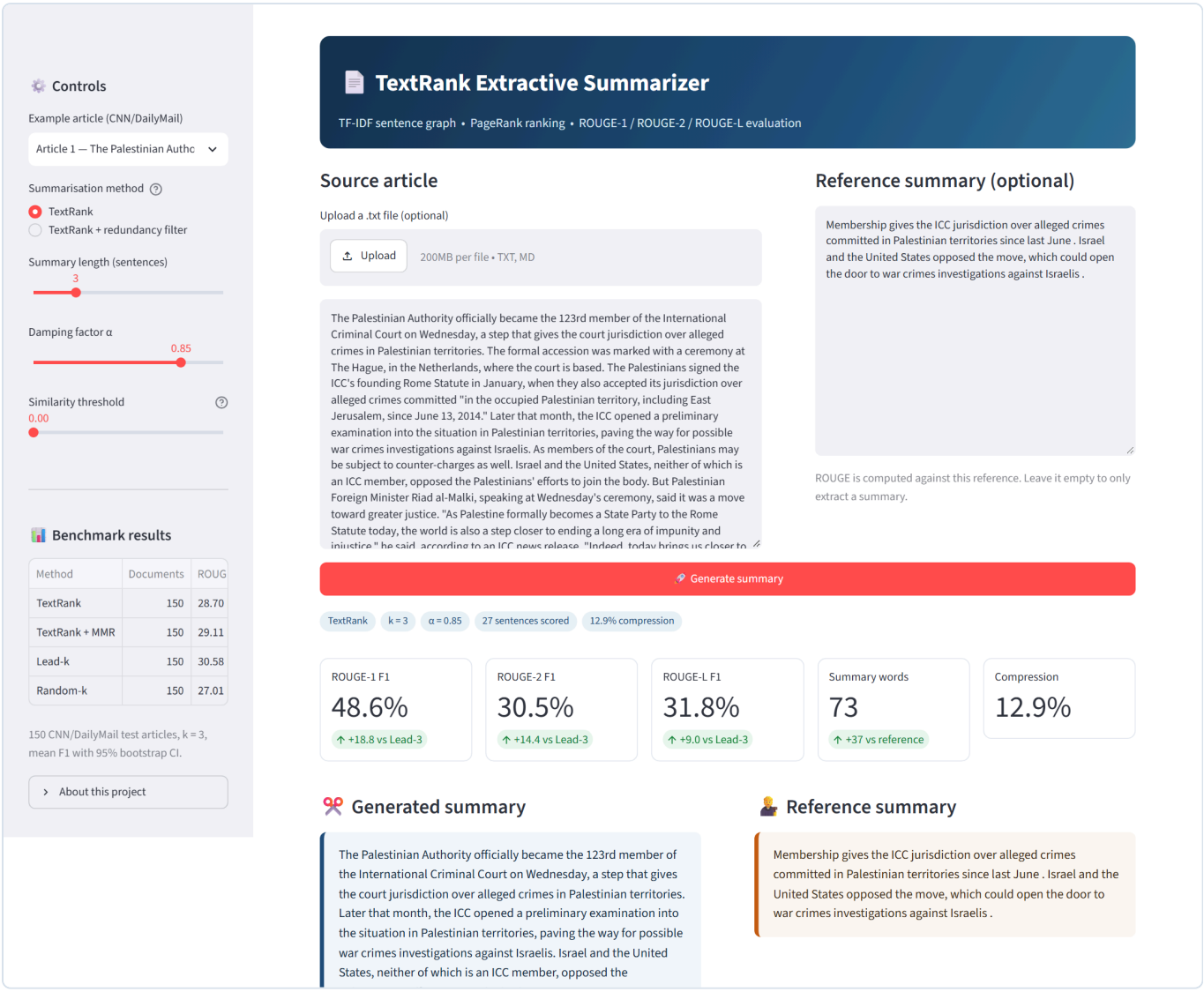


Figure 2. Complete application page, including the input panels, summary outputs and explainability tabs.

1. INTERACTIVE DEMO (STREAMLIT)

Full application page (part 2 of 2)

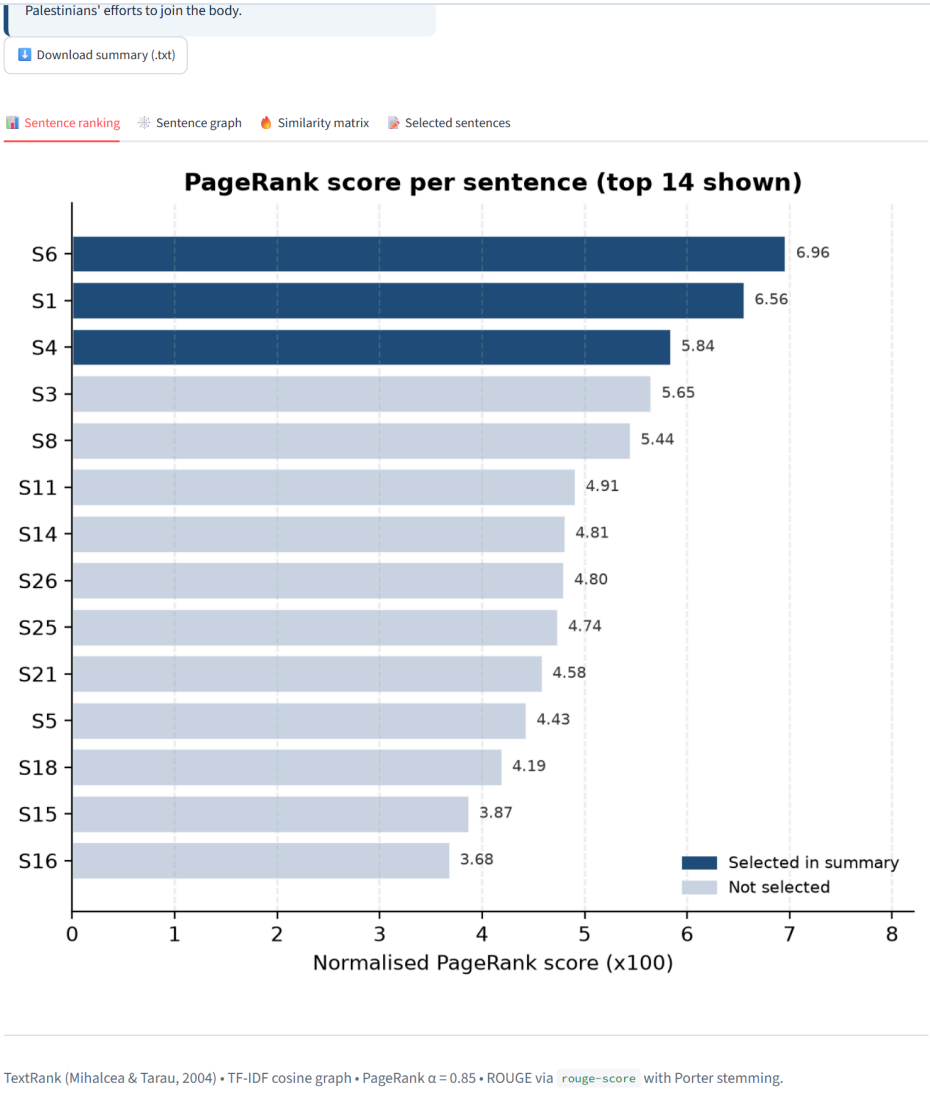


Figure 2. Complete application page, including the input panels, summary outputs and explainability tabs. (continued)

1. INTERACTIVE DEMO (STREAMLIT)

PageRank sentence ranking (part 1 of 2)

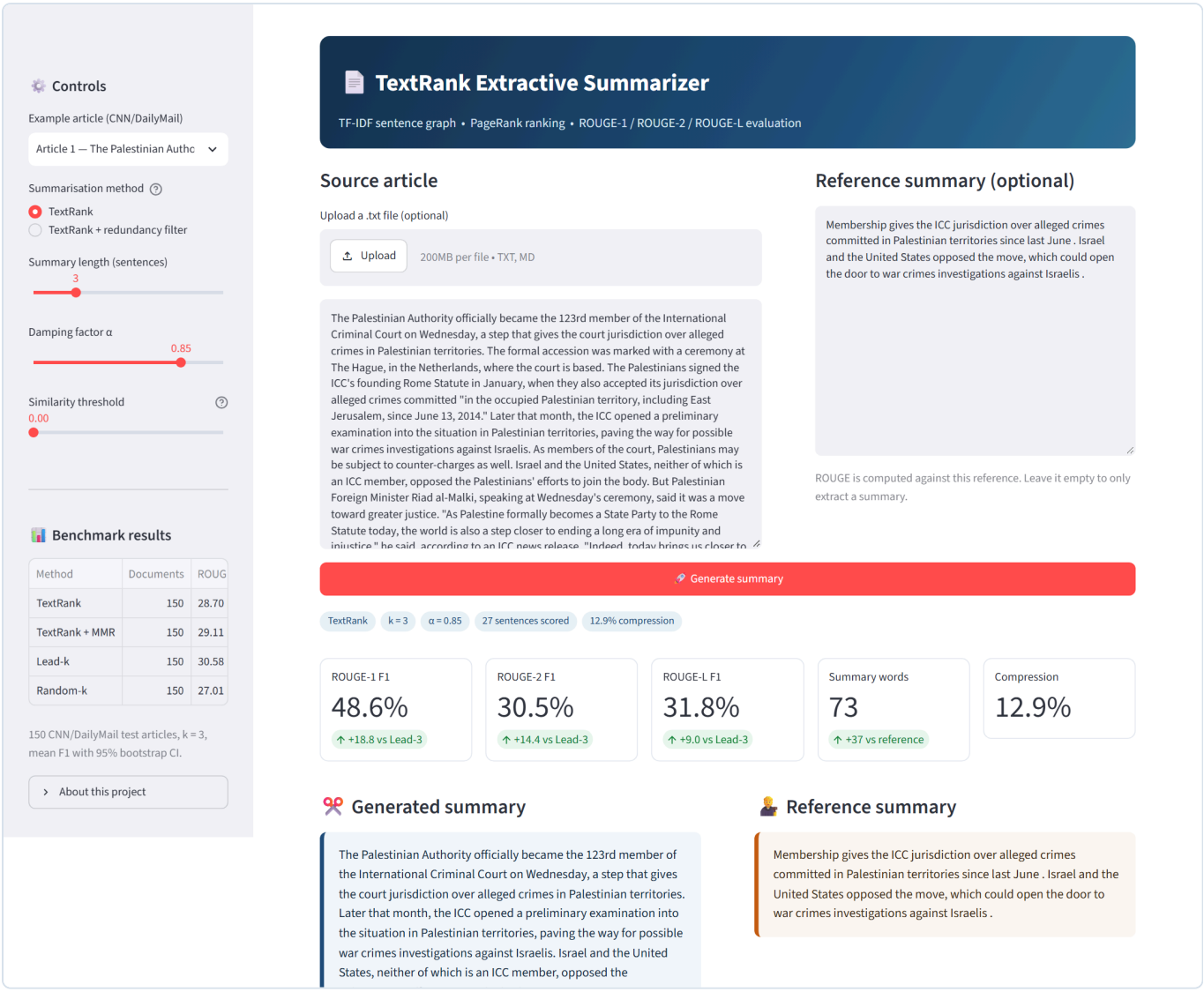


Figure 3. PageRank score for each sentence. Bars in dark blue are selected in the summary; the top-14 sentences are shown.

1. INTERACTIVE DEMO (STREAMLIT)

PageRank sentence ranking (part 2 of 2)

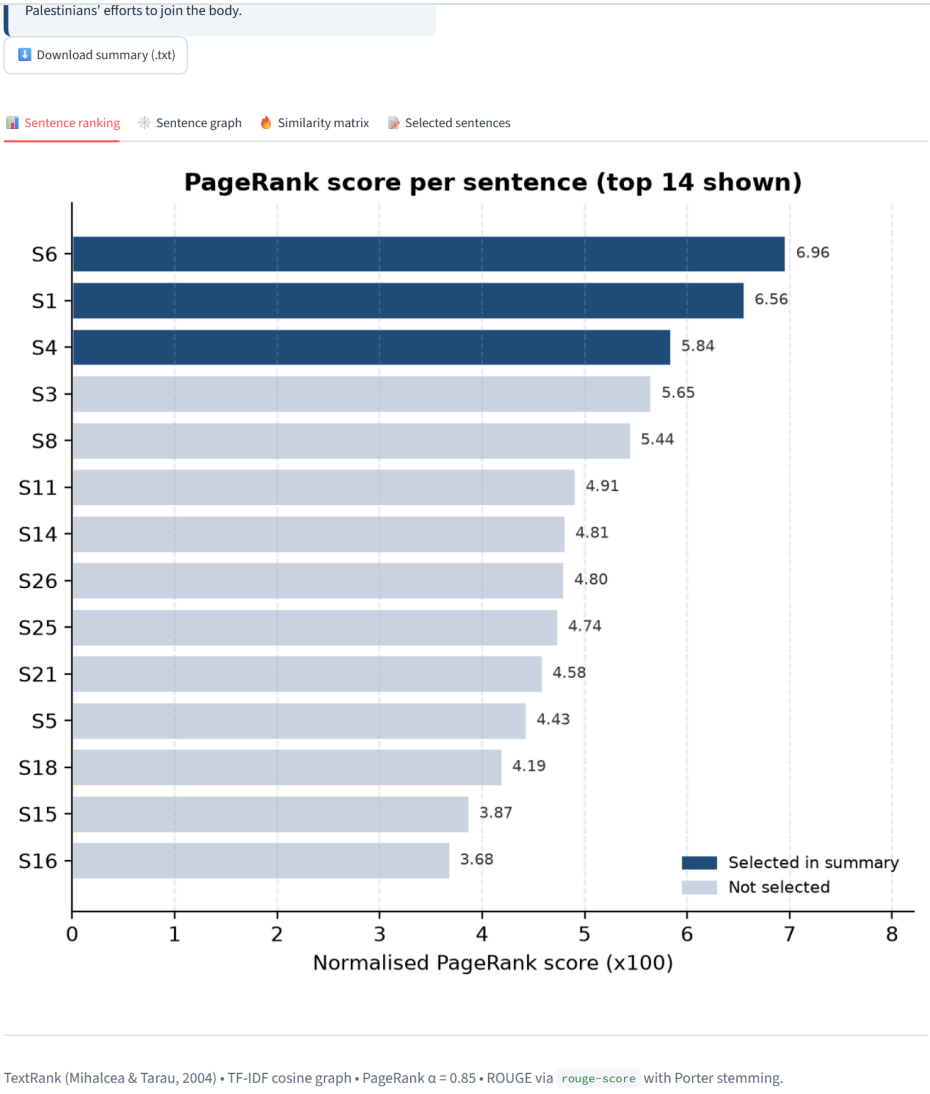


Figure 3. PageRank score for each sentence. Bars in dark blue are selected in the summary; the top-14 sentences are shown. (continued)

1. INTERACTIVE DEMO (STREAMLIT)

Sentence similarity graph (part 1 of 2)

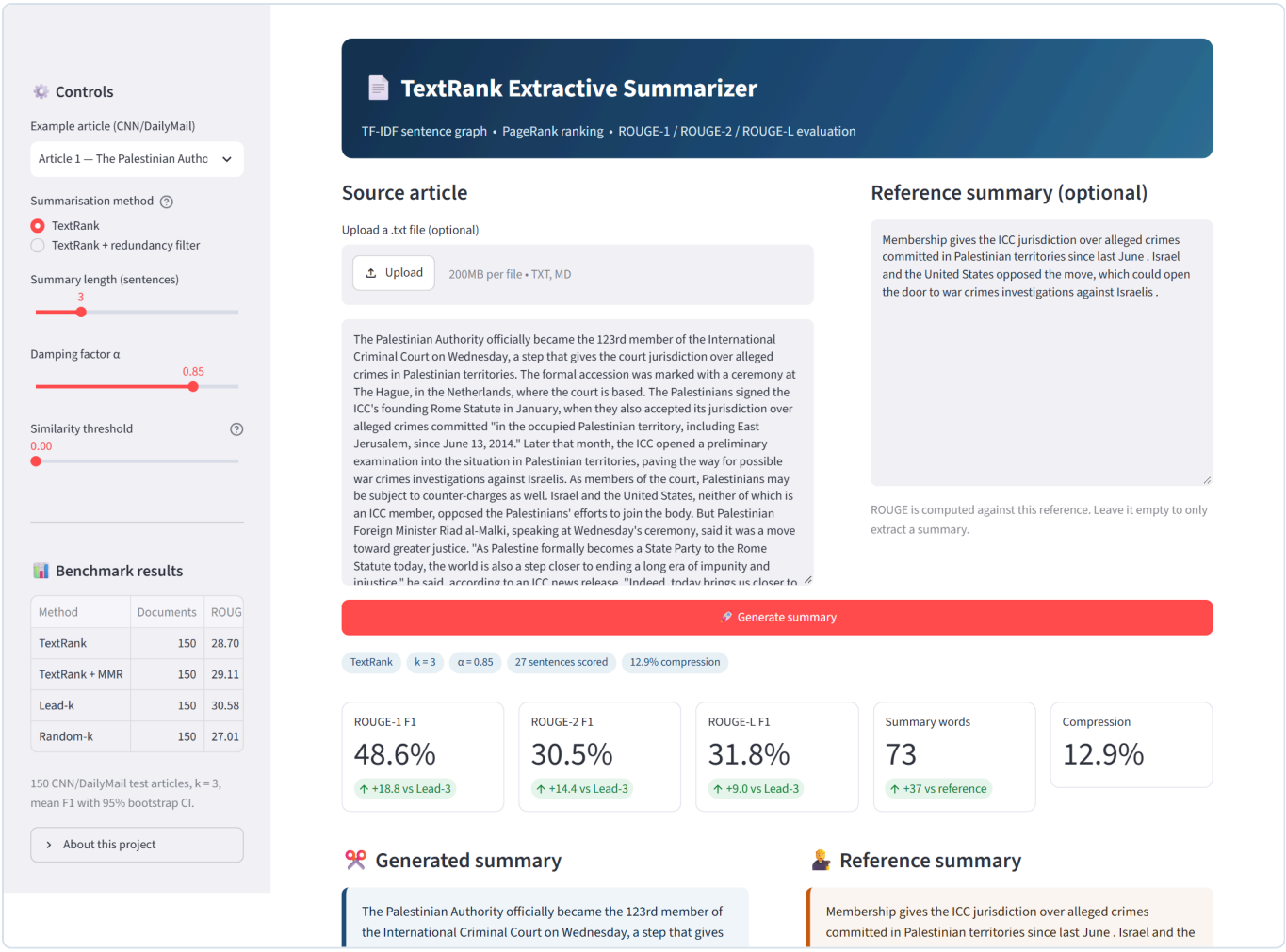


Figure 4. Spring-layout visualisation of the sentence graph. Node size and colour encode PageRank score; hub sentences are the ones TextRank selects.

1. INTERACTIVE DEMO (STREAMLIT)

Sentence similarity graph (part 2 of 2)

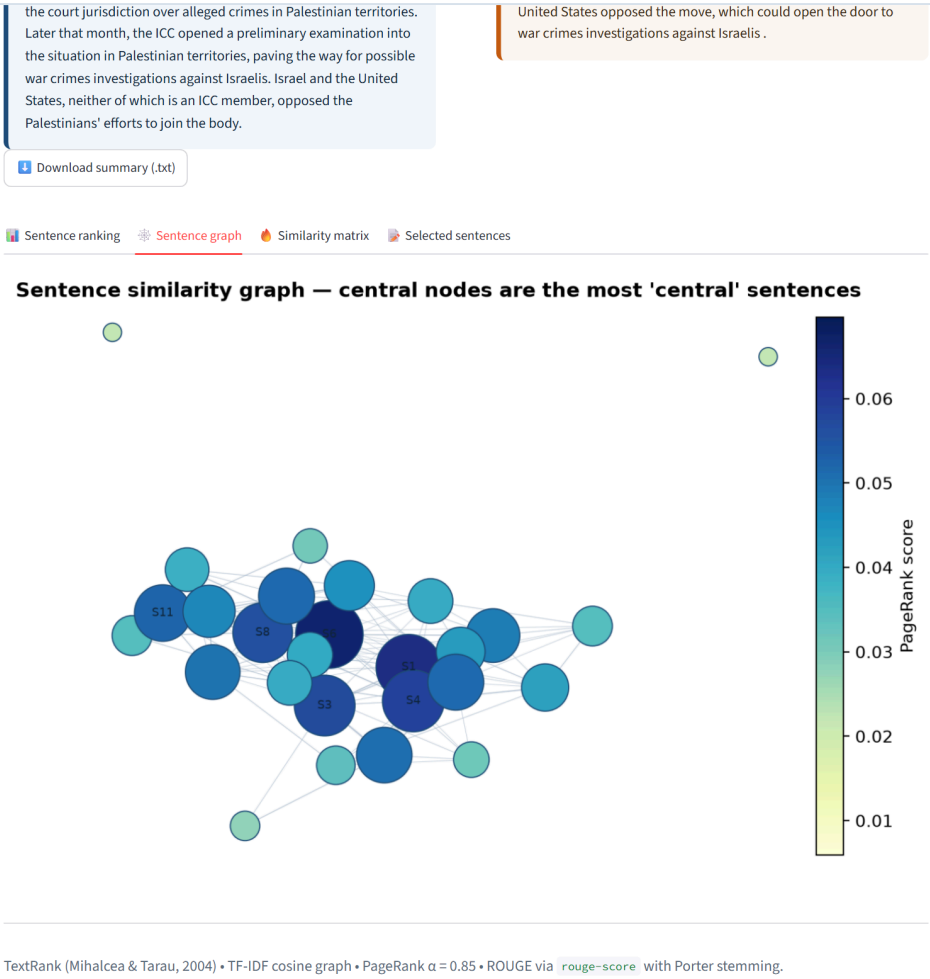


Figure 4. Spring-layout visualisation of the sentence graph. Node size and colour encode PageRank score; hub sentences are the ones TextRank selects. (continued)

1. INTERACTIVE DEMO (STREAMLIT)

Similarity matrix (part 1 of 2)

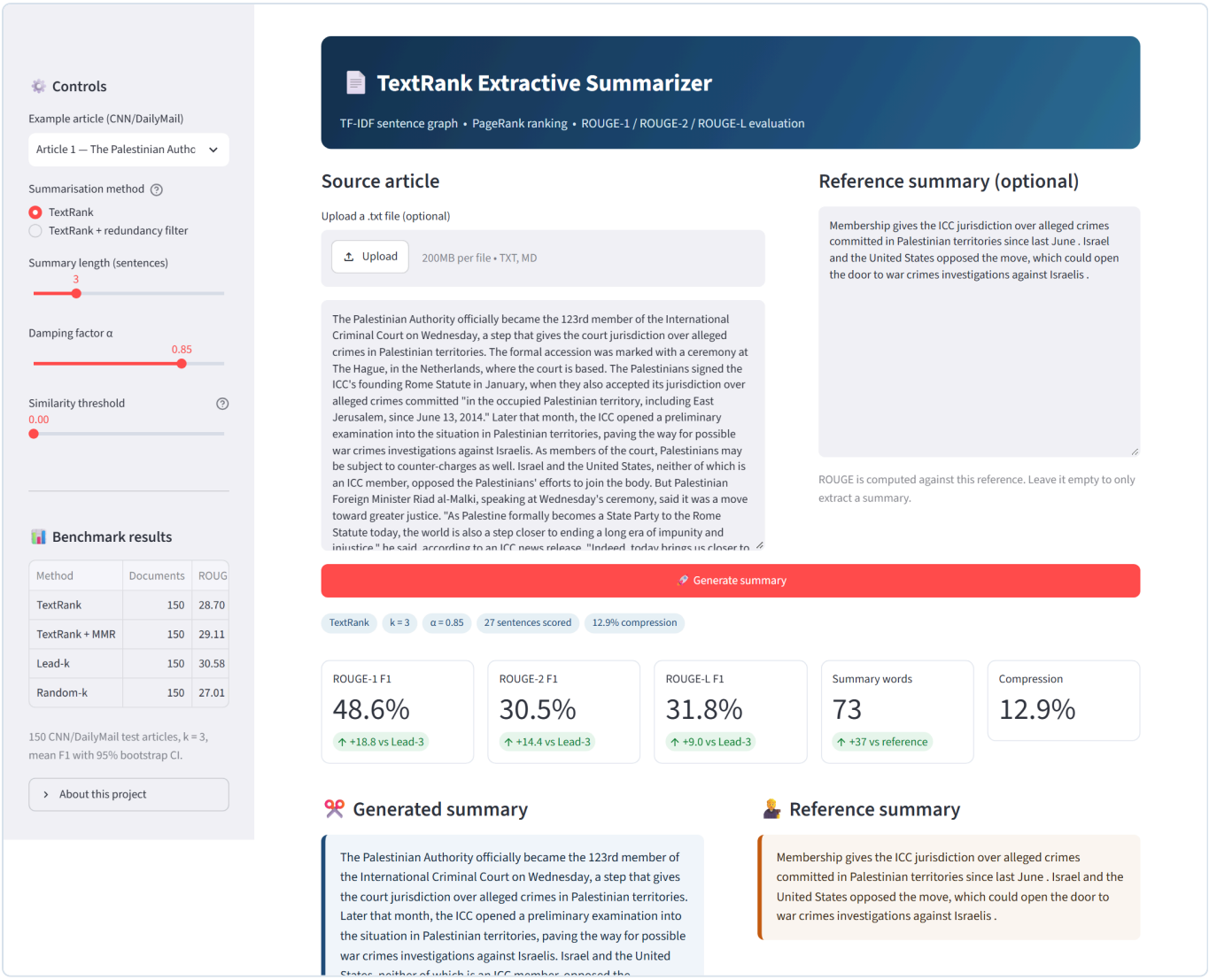


Figure 5. TF-IDF cosine similarity between sentence pairs for the selected article.

1. INTERACTIVE DEMO (STREAMLIT)

Similarity matrix (part 2 of 2)

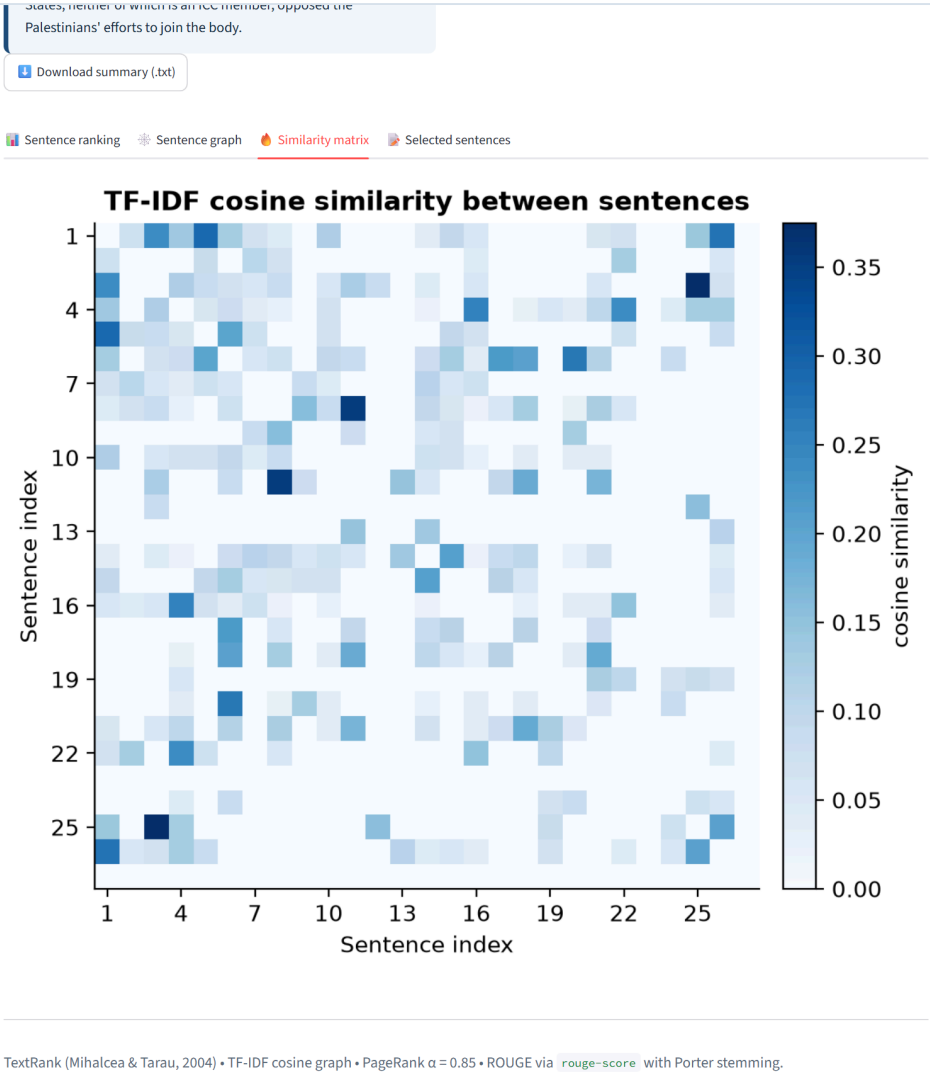


Figure 5. TF-IDF cosine similarity between sentence pairs for the selected article. (continued)

1. INTERACTIVE DEMO (STREAMLIT)

Selected sentences table (part 1 of 2)

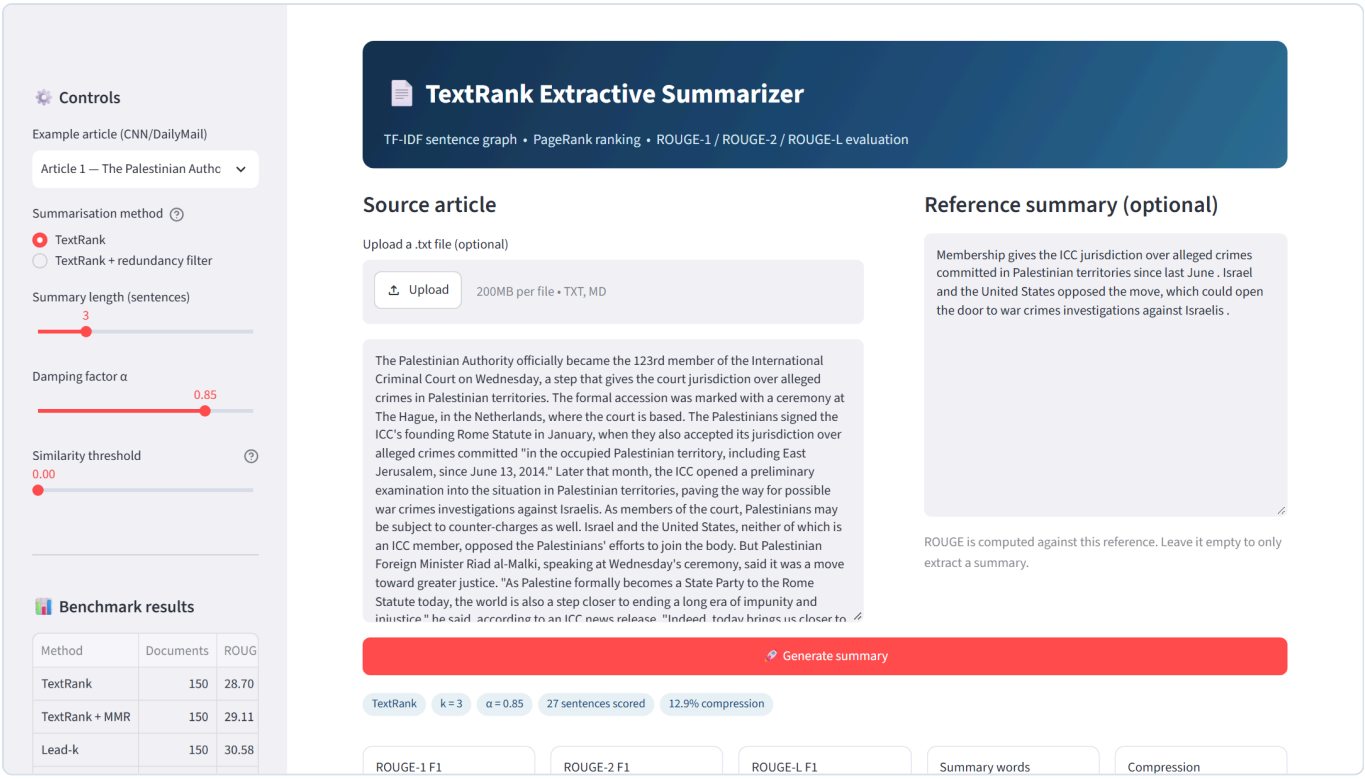


Figure 6. Selected sentences with their document position, PageRank score and extraction rank.

Selected sentences table (part 2 of 2)

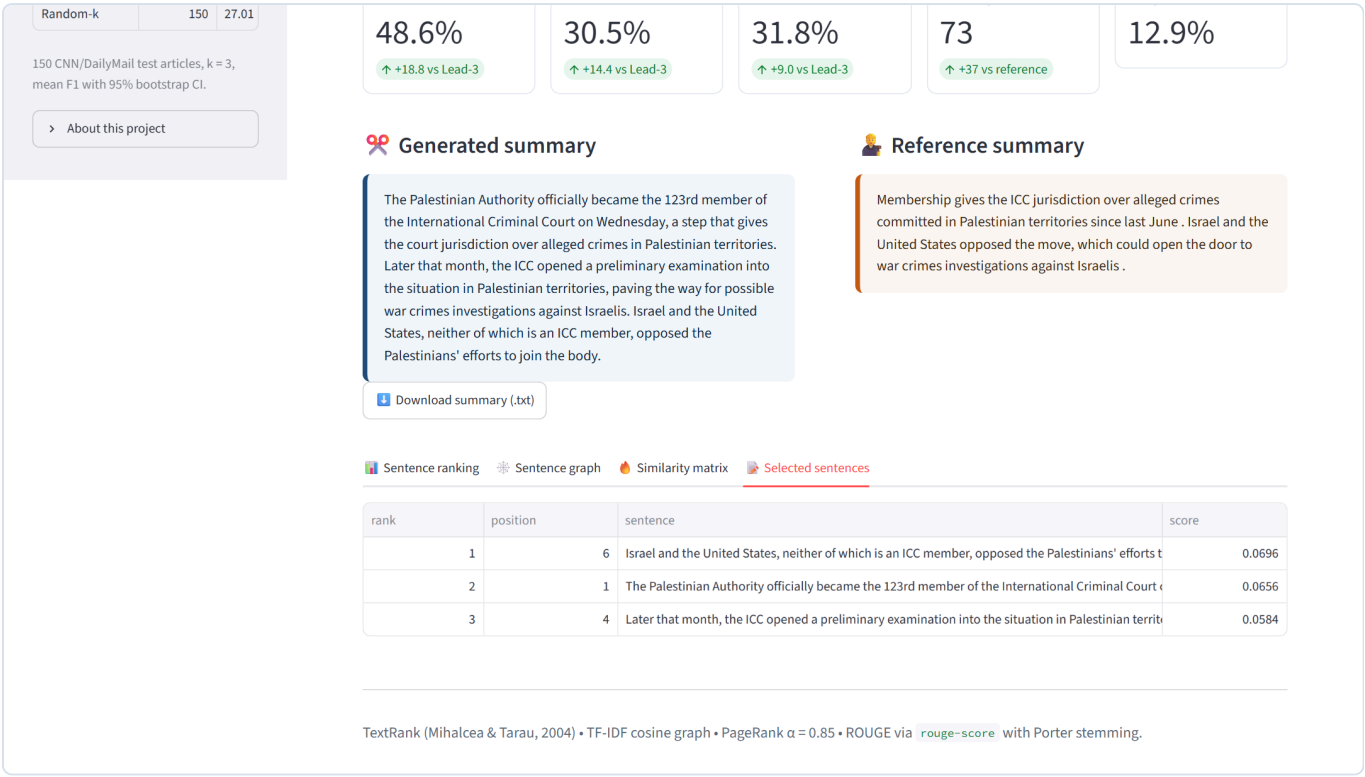


Figure 6. Selected sentences with their document position, PageRank score and extraction rank. (continued)

2. NOTEBOOK OUTPUTS

Notebook title and objective

Extractive Text Summarisation with TextRank

A TF-IDF sentence-graph approach evaluated with ROUGE-1/2/L

Mini Project — Natural Language Processing

Student	*Your Full Name*
Roll number	*Your Roll Number*
Course / semester	_Natural Language Processing — Semester ___
Guide	*Guide Name*
Institution	*Your Institution Name*
Date	*October 2026*

> **Objective.** Build an unsupervised extractive summariser that ranks sentences with TextRank, and quantify its quality against human-written reference summaries using ROUGE-1, ROUGE-2 and ROUGE-L. TextRank is compared with two baselines — the first *k* sentences (Lead-*k*) and a random-*k* extractor — so that the evaluation measures real algorithmic value rather than the trivial "news leads are informative" effect.

Deliverables covered by this notebook: the complete implementation, the experimental evaluation, all figures, and the evidence used in the written report and demo video.

Contents

1. Environment setup
2. Dataset: CNN/DailyMail evaluation slice
3. Text preprocessing
4. Sentence representation and similarity
5. Graph construction and PageRank
6. Summary extraction
7. Evaluation protocol and baselines
8. Corpus-level ROUGE results
9. Length sweep, ablation and error analysis

Figure 7. The executed notebook opens with the project title, student details and objective.

2. NOTEBOOK OUTPUTS

Preprocessing output

3. Text preprocessing

Sentence segmentation. Articles are split with NLTK's Punkt tokenizer, using the model bundled in `data/nltk_data` so no download is needed. Sentences with fewer than four tokens are discarded (they are usually captions, datelines or fragments), and exact duplicates are removed — they would otherwise inflate similarity and produce redundant summaries.

Tokenisation and normalisation. Each sentence is lowercased and tokenised with a regular expression that keeps words, hyphenated compounds and numbers. Stopwords (the scikit-learn English list plus a few news-boilerplate terms such as *said* and *also*) are removed, short tokens are dropped, and the remaining tokens are Porter-stemmed so that *election*, *elections* and *elect* share one dimension.

The result is a list of sentences and, for each sentence, a list of content tokens. Those token lists are the input to TF-IDF.

In [3]:
```
example = documents[0]
document = preprocess(example["article"])

preview = pd.DataFrame(
    {
        "sentence_id": range(1, len(document.sentences) + 1),
        "sentence": document.sentences,
        "words": [len(sentence.split()) for sentence in document.sentences],
        "content_tokens": [" ".join(tokens[:9]) + (" …" if len(tokens) > 9 else "") for tokens in document.tokens],
        "n_tokens": [len(tokens) for tokens in document.tokens],
    }
)
print(f"Kept {len(document.sentences)} sentences and {sum(len(t) for t in document.tokens)} content tokens.")
print(f"Original article: {count_words(example['article'])} words.")
display(preview.head(10))
```

```
Kept 27 sentences and 303 content tokens.
Original article: 567 words.
```

	sentence_id	sentence	words	content_tokens	n_tokens
0	1	The Palestinian Authority officially became the 123rd member of the International Criminal Court on Wednes...	28	palestinian author offici 123 rd member intern crimin court …	18
1	2	The formal accession was marked with a ceremony at The Hague, in the Netherlands, where the court is based.	19	formal access mark ceremoni hagu netherland court base	8
2	3	The Palestinians signed the ICC's founding Rome Statute in January, when they also accepted its jurisdicti...	32	palestinian sign icc' found rome statut januari accept jurisdict …	21
3	4	Later that month, the ICC opened a preliminary examination into the situation in Palestinian territories, ...	25	later month icc open preliminari examin situat palestinian territori …	16

Figure 8. Sentence segmentation and tokenisation output for the first article, including the number of sentences and content tokens.

2. NOTEBOOK OUTPUTS

Corpus-level ROUGE results

8. Corpus-level ROUGE results

The table above is the central result of the project. Three observations matter:

1. **TextRank clearly beats the random baseline** on all three metrics, which confirms that graph centrality captures something real about sentence importance.
2. **Lead-3 is stronger than TextRank on CNN/DailyMail.** This is not a bug: news articles follow an inverted-pyramid structure, so the first sentences really do contain the most summary-worthy content. Lead-3 is a notoriously hard baseline on this dataset, and reporting it honestly is part of a sound evaluation.
3. **Redundancy filtering gives a small, non-significant improvement.** The documents are short enough that near-duplicate sentences are rare, so there is little redundancy for the filter to remove.

The cell below restates the comparison numerically and reports the paired significance tests.

In [8]:

```
summary = results["summary_k3"].set_index("method")
comparisons = pd.DataFrame(results["comparisons"])
comparisons["mean_difference_pp"] = comparisons["mean_difference"] * 100
comparisons["ci_low_pp"] = comparisons["ci_low"] * 100
comparisons["ci_high_pp"] = comparisons["ci_high"] * 100

table = comparisons[
    ["method_a", "method_b", "metric", "mean_a", "mean_b", "mean_difference_pp",
     "ci_low_pp", "ci_high_pp", "a_wins", "b_wins", "p_value"]
].copy()
table[["mean_a", "mean_b"]] = (table[["mean_a", "mean_b"]] * 100).round(2)
table[["mean_difference_pp", "ci_low_pp", "ci_high_pp"]] = table[
    ["mean_difference_pp", "ci_low_pp", "ci_high_pp"]
].round(2)
table["p_value"] = table["p_value"].map(lambda value: f"{value:.4f}" if pd.notna(value) else "n/a")
display(
    table.rename(
        columns={
            "method_a": "Method A", "method_b": "Method B", "metric": "Metric",
            "mean_a": "A (%)", "mean_b": "B (%)", "mean_difference_pp": "Difference (pp)",
            "ci_low_pp": "95% CI low", "ci_high_pp": "95% CI high",
            "a_wins": "A wins", "b_wins": "B wins", "p_value": "Wilcoxon p",
        }
    ).style.hide(axis="index")
)

textrank1 = summary.loc["TextRank", "rouge1_f1"] * 100
lead1 = summary.loc["Lead-k", "rouge1_f1"] * 100
```

Figure 9. The notebook computes ROUGE for every method on all 150 documents and renders the comparison table inline.

2. NOTEBOOK OUTPUTS

Length sweep table and figure

9. Length sweep, ablation and error analysis

A single operating point can hide a lot. The sweep below varies the number of extracted sentences from 2 to 8 and asks whether the ranking of methods is stable. Two effects usually appear:

- **All methods peak at k = 2–3.** As *k* grows, precision falls faster than recall rises, so ROUGE-1 F1 declines for every method and the ordering of methods stays stable.
- **TextRank beats Random-3 at every length.** The graph ranking is consistently more informative than content-blind selection, even though it never overtakes the news-specific Lead baseline.

The per-document comparison then shows *where* TextRank wins: not uniformly, but on articles whose key information is not confined to the opening sentences (background pieces, multi-topic reports, articles with an anecdotal lead).

In [9]:
```
sweep = results["metrics_by_method_k"]
pivot = sweep.pivot_table(index="k", columns="method", values="rouge1_f1").round(3)
display(Markdown("**ROUGE-1 F1 by summary length**"))
display(pivot)

fig = visualize.plot_metric_by_length(sweep, ROOT / "results" / "figures" / "rouge_by_length.png")
plt.close(fig)
display(Image(filename=str(ROOT / "results" / "figures" / "rouge_by_length.png"), width=720))

display(Markdown("**Precision–recall trade-off (all methods, k = 3)**"))
tradeoff = results["summary_k3"][
    ["method", "rouge1_precision", "rouge1_recall", "rouge1_f1", "summary_words", "compression_ratio"]
].copy()
tradeoff[["rouge1_precision", "rouge1_recall", "rouge1_f1"]] *= 100
tradeoff["compression_ratio"] = (tradeoff["compression_ratio"] * 100).round(1)
display(tradeoff.round(2).rename(columns={
    "rouge1_precision": "R-1 precision (%)", "rouge1_recall": "R-1 recall (%)",
    "rouge1_f1": "R-1 F1 (%)", "summary_words": "Summary words", "compression_ratio": "Compression (%)",
}).style.hide(axis="index"))
```

ROUGE-1 F1 by summary length

method	Lead-k	Random-k	TextRank	TextRank + MMR
k				
2	0.313	0.250	0.297	0.301
3	0.306	0.270	0.287	0.291
4	0.298	0.256	0.269	0.271

Figure 10. ROUGE-1 F1 as the summary length varies, produced during notebook execution.

3. EVALUATION RESULTS

ROUGE results table (k = 3, n = 150)

Method	Run label	N	ROUGE-1 F1	ROUGE-2 F1	ROUGE-L F1	Words	Compression
TextRank	TextRank	150	28.70 [27.10-30.36]	10.13 [8.80-11.52]	19.75 [18.46-21.02]	70	18.5%
TextRank + MMR	TextRank + MMR	150	29.11 [27.53-30.70]	10.33 [8.99-11.72]	20.04 [18.80-21.32]	70	18.3%
Lead-3	Lead-k	150	30.58 [28.69-32.51]	12.53 [10.92-14.19]	21.19 [19.64-22.77]	66	17.5%
Random-3	Random-k	150	27.01 [25.36-28.68]	8.29 [7.03-9.69]	18.26 [16.94-19.64]	60	16.1%

Table 1. Mean ROUGE F1 with 95% bootstrap confidence intervals, saved by the experiment runner as `results/results_table_k3.csv`. The table shows the canonical run labels (`Lead-k`, `Random-k`); at k = 3 these correspond to Lead-3 and Random-3.

3. EVALUATION RESULTS

ROUGE F1 by method

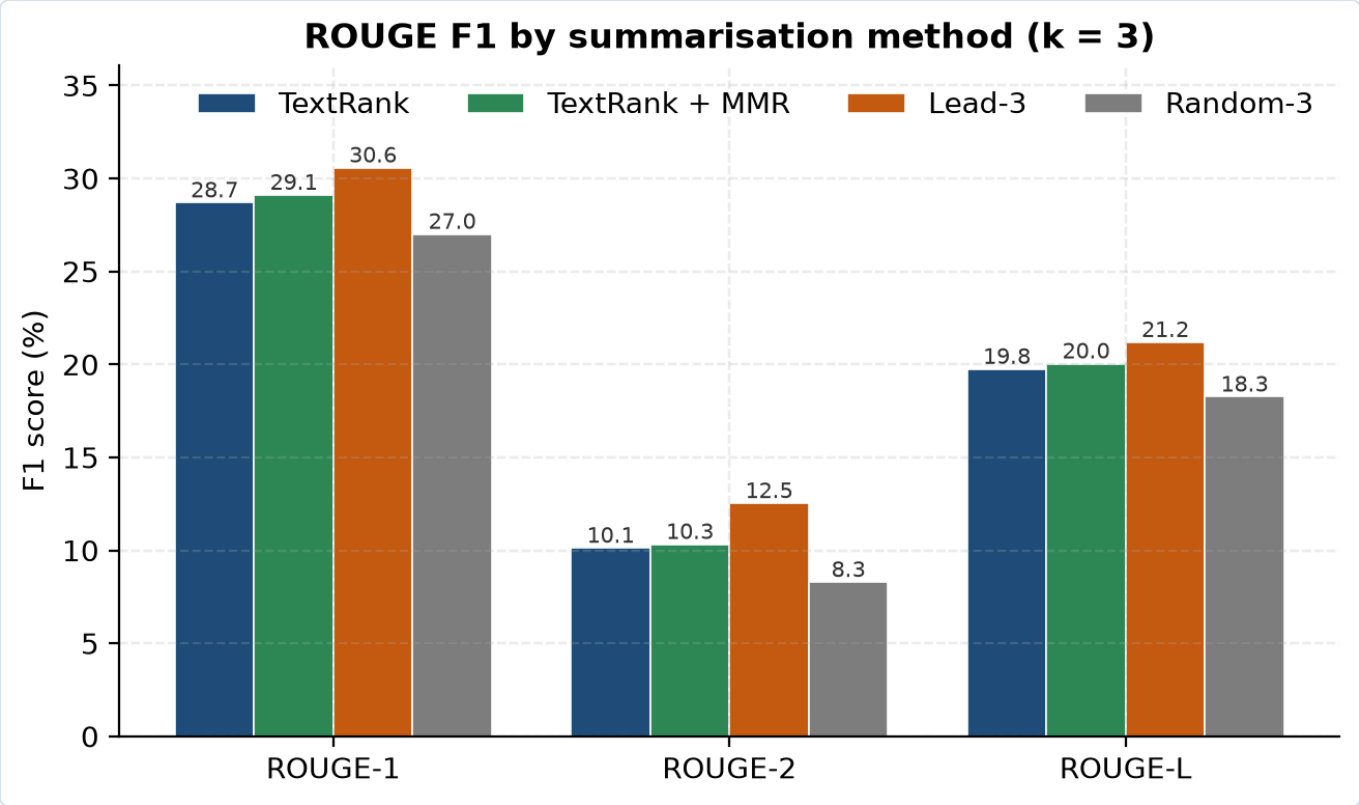


Figure 11. ROUGE-1, ROUGE-2 and ROUGE-L F1 for TextRank, the redundancy-filtered variant, Lead-3 and Random-3.

Per-document comparison against Lead-3

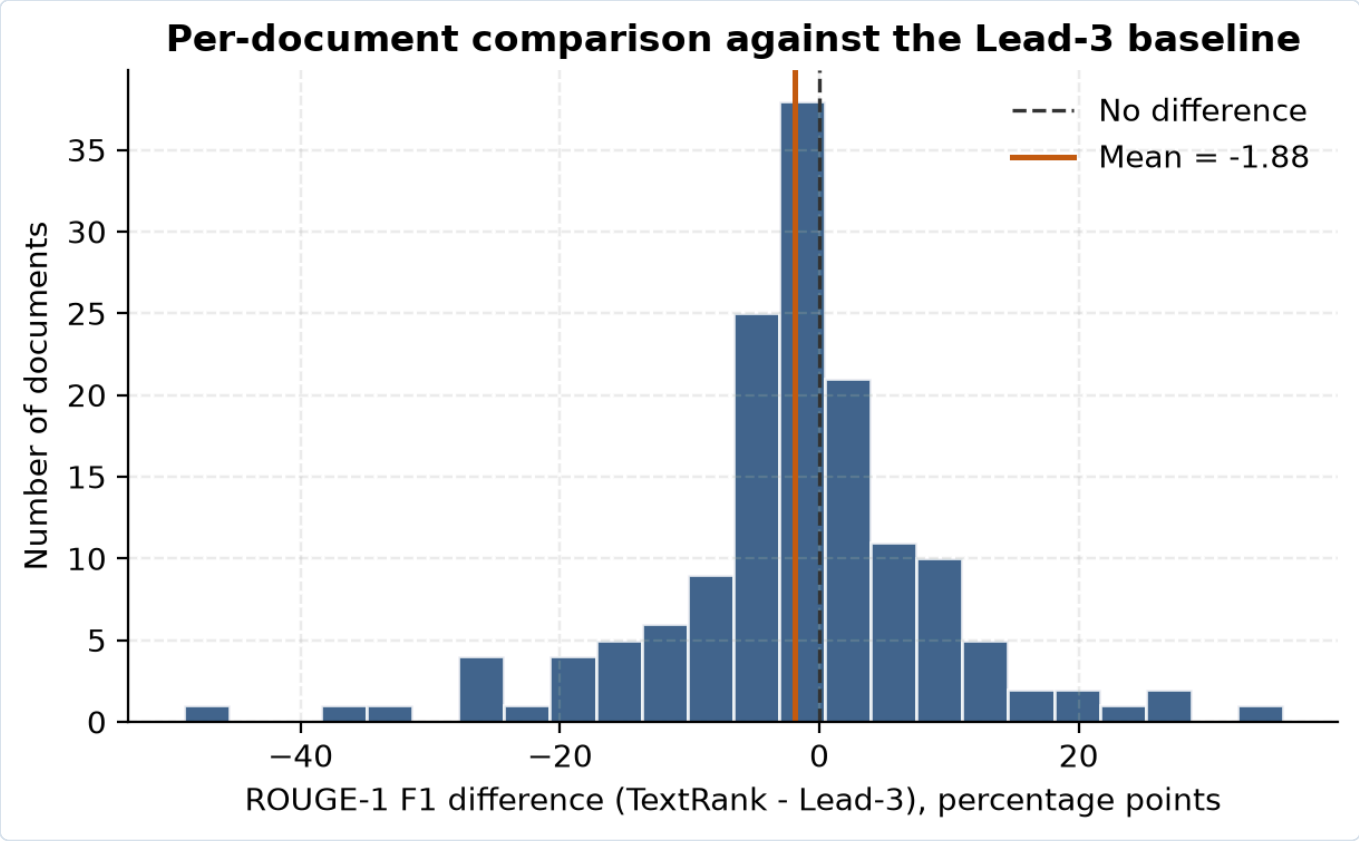


Figure 12. Distribution of per-document ROUGE-1 F1 differences between TextRank and Lead-3, with the mean difference marked.

3. EVALUATION RESULTS

Effect of summary length

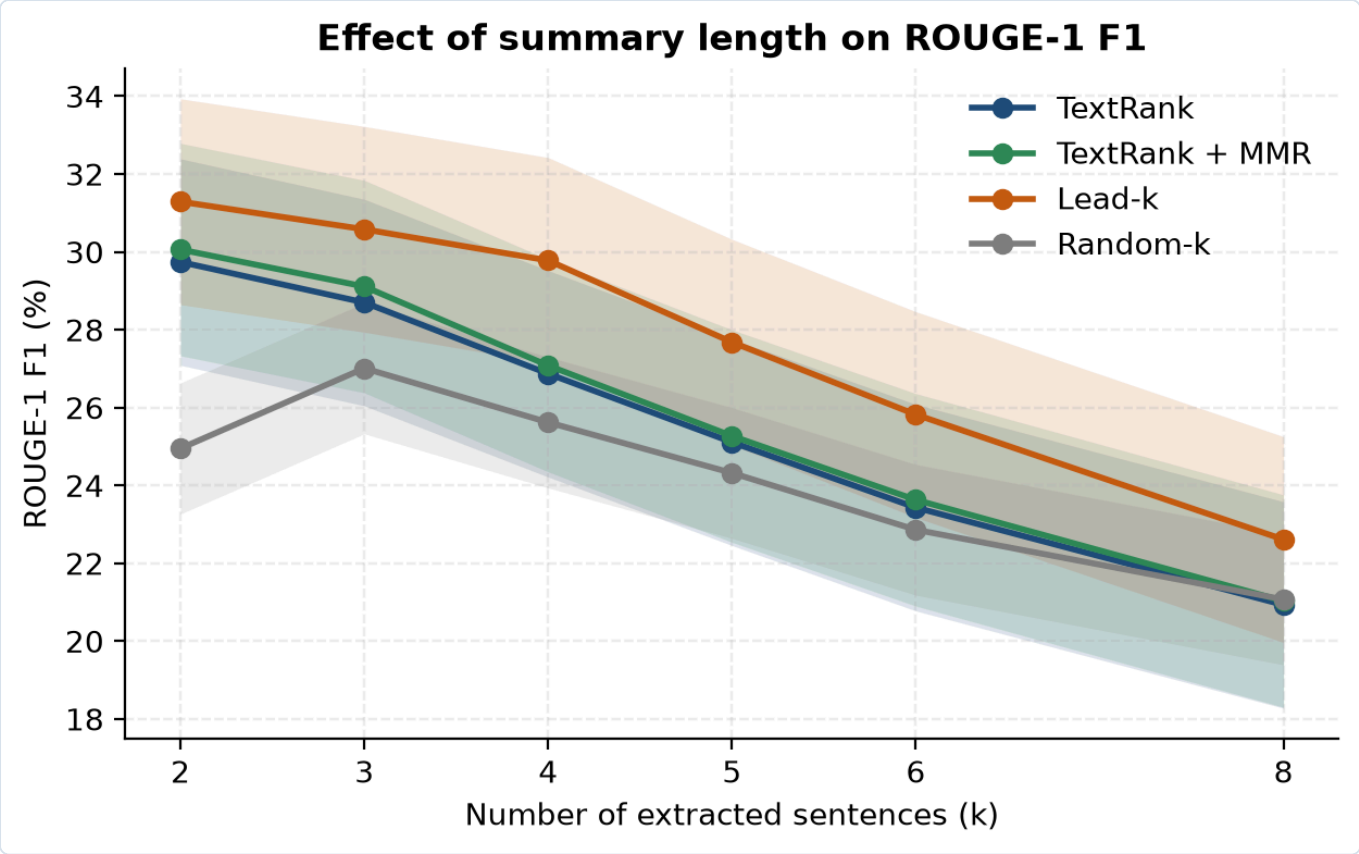


Figure 13. ROUGE-1 F1 versus summary length with 95% confidence bands for all four methods.

3. EVALUATION RESULTS

Redundancy-filter ablation

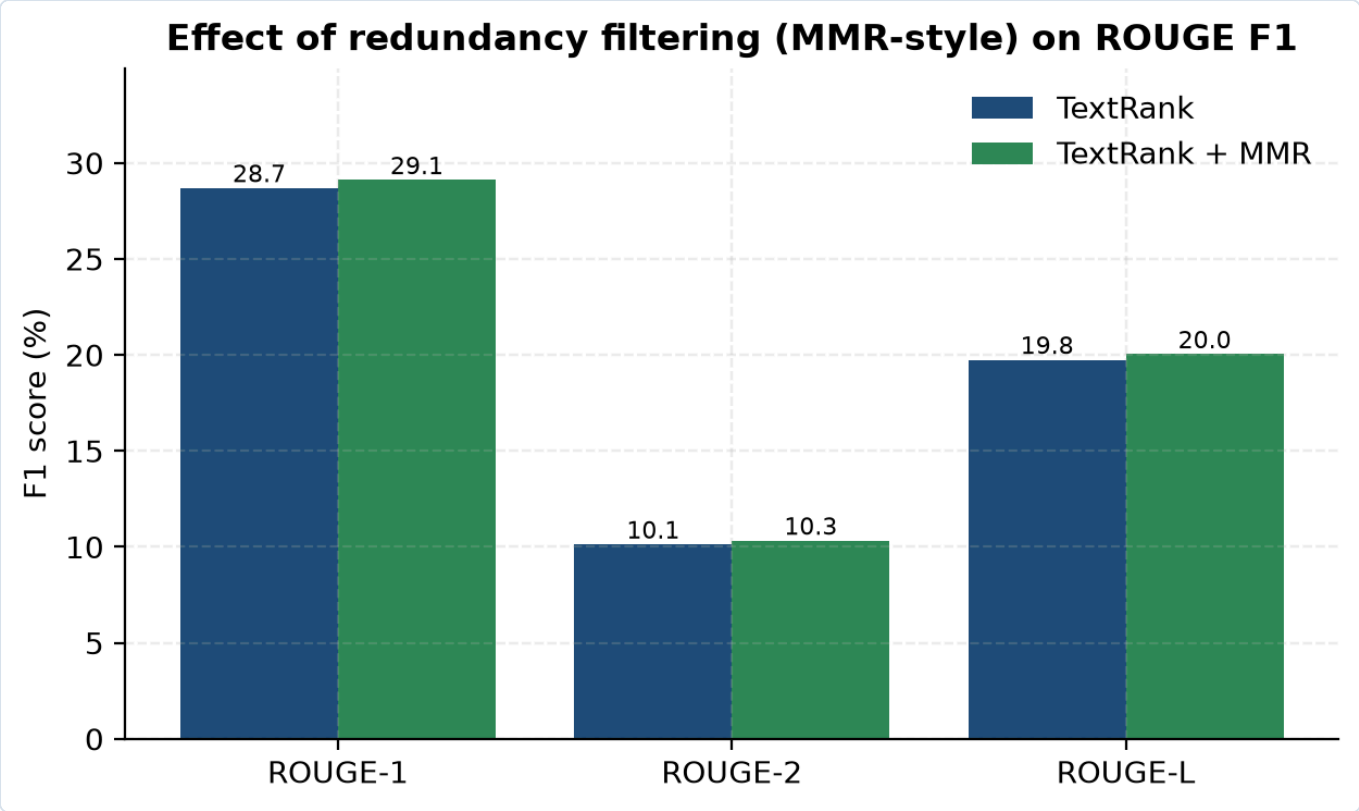


Figure 14. Ablation at k = 3: plain TextRank versus TextRank with the MMR-style redundancy filter.

Sentence graph of an example article

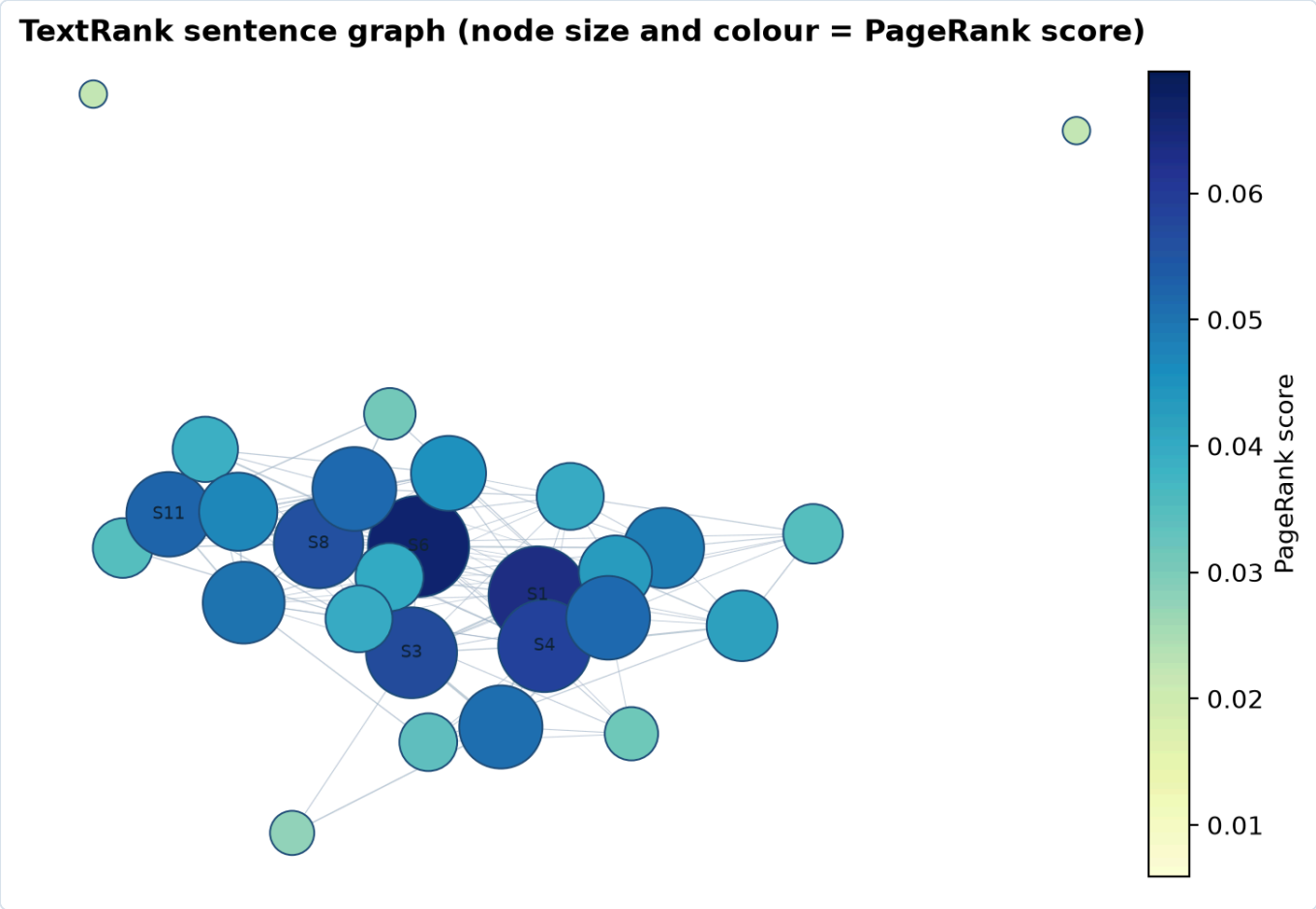


Figure 15. Static rendering of the TextRank sentence graph used in the report; node colour and size encode PageRank score.

4. REPRODUCIBILITY

Terminal output of the full evaluation

```
$ python scripts/run_experiment.py

processed 125/150 documents

processed 150/150 documents



=== ROUGE F1 at k = 3 (mean [95% bootstrap CI]) ===

        Method  Documents             ROUGE-1             ROUGE-2             ROUGE-L Avg. words
Compression

      TextRank        150 28.70 [27.10-30.36]  10.13 [8.80-11.52] 19.75 [18.46-21.02]         70
18.5%

TextRank + MMR        150 29.11 [27.53-30.70]  10.33 [8.99-11.72] 20.04 [18.80-21.32]         70
18.3%

        Lead-k        150 30.58 [28.69-32.51] 12.53 [10.92-14.19] 21.19 [19.64-22.77]         66
17.5%

      Random-k        150 27.01 [25.36-28.68]    8.29 [7.03-9.69] 18.26 [16.94-19.64]         60
16.1%



Saved 3600 predictions in 7.0s -> C:\Users\srira\textrank-summarizer\results
```

Figure 16. Console output of the experiment runner: 3,600 summaries (150 documents × 6 lengths × 4 methods) evaluated in a few seconds, followed by the ROUGE table that is written to `results/`.

REPOSITORY SNAPSHOT

```
textrank-summarizer/
├── data/cnn_dailymail_sample.json     150 CNN/DailyMail articles + references
├── notebooks/textrank_evaluation.ipynb  executed notebook (27 cells)
├── src/                               pipeline, baselines, ROUGE, app
├── scripts/                           data, experiments, report, screenshots
├── results/                           CSVs, metrics, figures, metadata
├── report/                            report.md, report.html, student.json
└── deliverables/                      project_report.pdf, this PDF
```